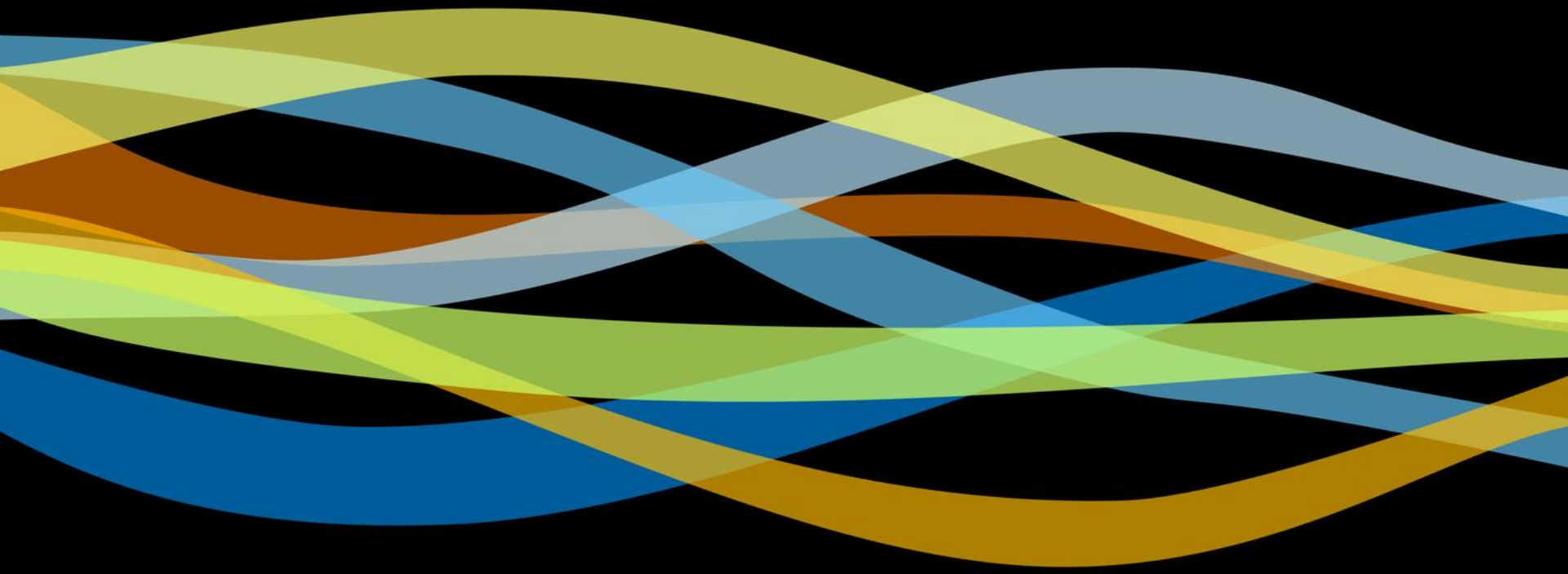


LINUX_

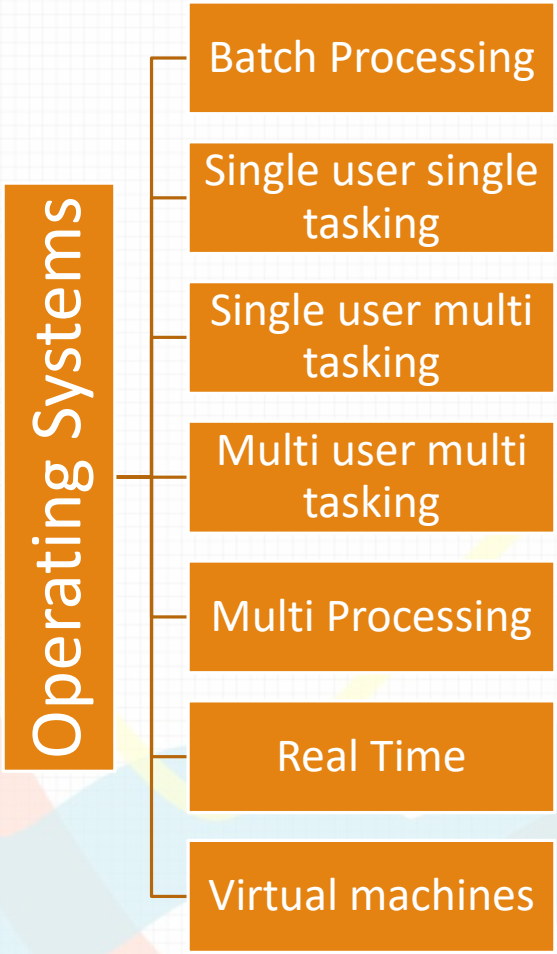
Operating System

A collection of sysytems programs that control the operations of the computer system.

Functions:

1. Manages the computer hardware
2. Provides a user interface
3. Process management
4. Memory Management
5. File Management
6. Security management.

Types of Operating Systems



Unix Operating System

Developed in 1969.

Ken Thompson, Dennis Ritchie, Brian Kernighan, Douglas McIlroy, Joe Ossanna, and other group of AT&T employees at Bell Labs created Unix

Under constant development

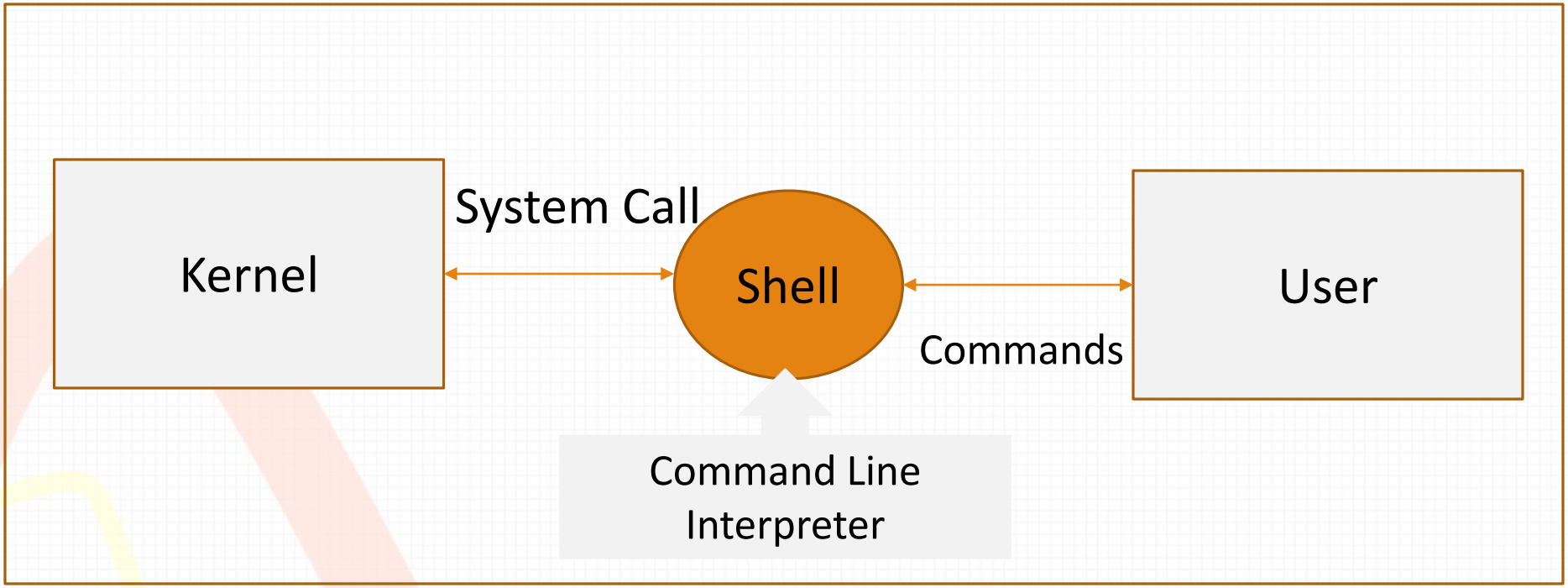
A stable , multi user , muti tasking operating system for servers, desktops and laptop computers.

Varieties: SunSolaris, GNU/Linux, MacOS X

Unix Components

1. **Kernel:**
   - The kernel of UNIX is the hub of the operating system: it allocates time and memory to programs and handles the filestore and communications in response to system calls.
2. **Shell:**
   - The shell acts as an interface between the user and the kernel. BASH (Bourne-Again SHell), CSH (C SHell), KSH (Korn SHell), TCSH

3. **Programs:** Commands are Programs



File

Collection of Data

Process

Executing program identified by a unique PID

Introduction to Linux

Linux is a UNIX-based operating system. Its original creator was a Finnish student name Linus Torvalds.

Linux is an open source and community-developed operating_system for computers, servers, mainframes, mobile devices and embedded devices. It is supported on almost every major computer platform including x86, ARM and SPARC, making it one of the most widely supported operating systems.

The Linux OS is frequently packaged as a Linux distribution for both desktop and server use, and includes the Linux kernel (the core of the operating system) as well as supporting tools and libraries. Popular Linux OS distributions include Debian, Ubuntu, Fedora, Red Hat and openSUSE.

Advantage of linux

- Low cost
- Stability
- Performance
- Networking
- Flexibilty
- Compatibilty
- Multitasking
- Security

Disadvantage of linux

- **Understanding** – Becoming familiar with the Linux operating system requires patience as well as a strong learning curve. You must have the desire to read and figure things out on your own, rather than having everything done for you.

- **Compatibility** – Because of its free nature, Linux is sometimes behind the curve when it comes to brand new hardware compatibility. Though the kernel contributors and maintainers work hard at keeping the kernel up to date, Linux does not have as much of a corporate backing as alternative operating systems. Sometimes you can find third party applications, sometimes you can't.

- **Alternative Programs** – Though Linux developers have done a great job at creating alternatives to popular Windows applications, there are still some applications that exist on Windows that have no equivalent Linux application.

Basic Commands of linux

- **ls -→ Listing File**

  If you want to see the list of files on your UNIX or Linux system, use the '**ls**' command.

  It shows the files /directories in your current directory.

```
guru99@VirtualBox:~$ ls
Desktop     Downloads          Music      Public      Videos
Documents   examples.desktop   Pictures   Templates
guru99@VirtualBox:~$
```

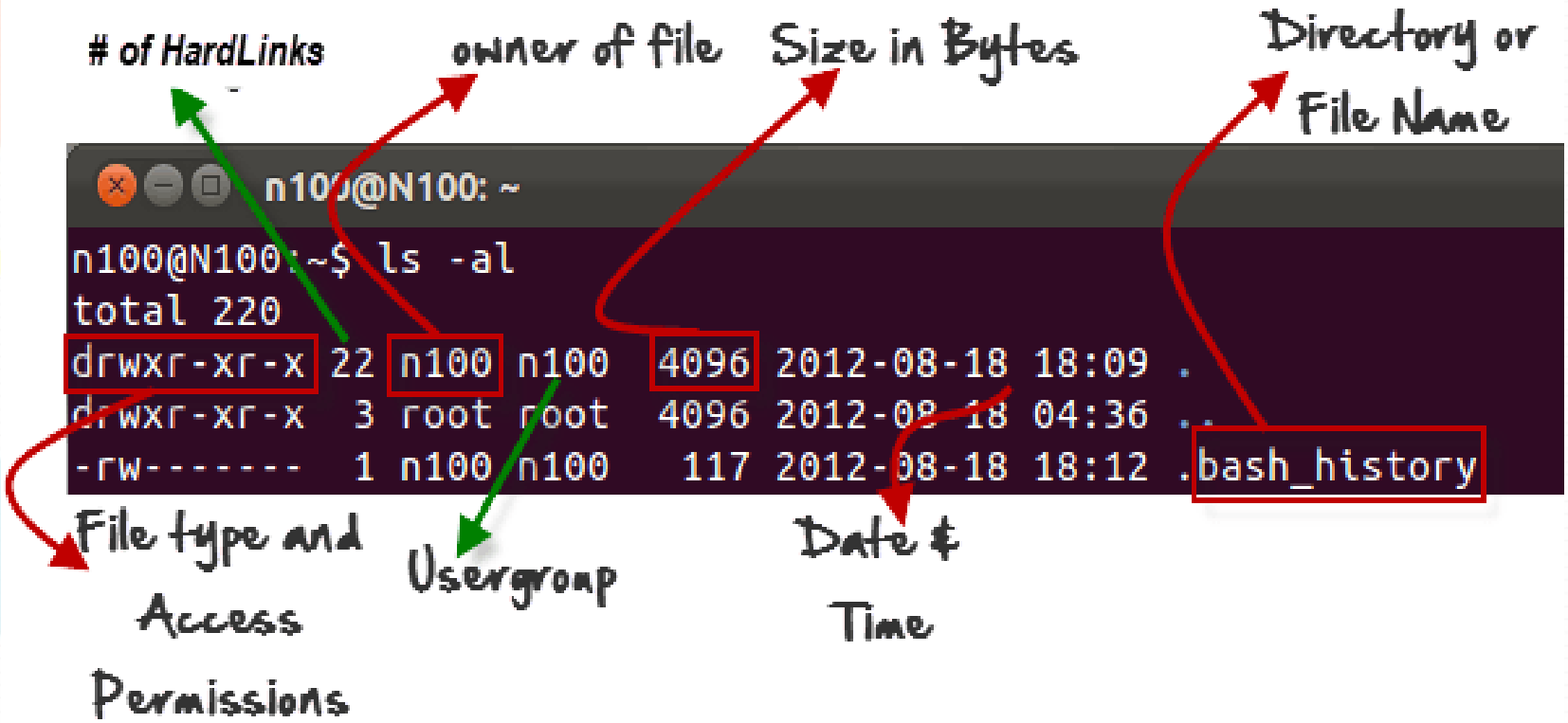
Note:-

- Directories are denoted in blue color.
- Files are denoted in white color.

- **ls -al→ Listing detailed information.**

  If you want to see the list of files and complete information(like permission, owner , user group etc.) of the file use the "**ls –al**" command.

  See example:-



- **ls -a→ Listing Hidden Files**

Hidden items in UNIX/Linux begin with - **. "period"** symbol at the start, of the file or directory.
Any Directory/file starting with a '.' will not be seen unless you request for it. To view hidden files, use the command **ls –a.**
Example:-

```
guru99@VirtualBox:~$ ls -a
.                .dmrc              .ICEauthority      sample
..               Documents          .local             sample1
.bash_history    Downloads          .mission-control   sample2
.bash_logout     examples.desktop   Music              Templates
.bashrc          .gconf             Pictures           .thumbnails
.cache           .gnome2            .profile           Videos
.config          .gstreamer-0.10    Public             .Xauthority
.dbus            .gtk-bookmarks     .pulse             .xsession-erro
Desktop          .gvfs              .pulse-cookie
guru99@VirtualBox:~$
```

Creating & Viewing Files

- The **'cat'** command is used to display text files. It can also be used for copying, combining and creating new text files. Let's see how it works.

- **To create a new file, use the command**
  - cat > filename
  - Add content
  - Press 'ctrl + d' to return to command prompt.

Create a File

```
guru99@VirtualBox:~$ cat > sample1
```

Enter Content

```
This is sample1
```

Press Control + D to exit

```
guru99@VirtualBox:~$
```

- To view a file, use the command – **cat filename**
- Let's see the file we just created –

```
guru99@VirtualBox:~$ cat sample1
This is sample1
```

Deleting Files

- The **'rm'** command removes files from the system without confirmation.
- To remove a file use syntax – **rm filename.**

List current contents of directory

```
guru99@VirtualBox:~$ ls
Desktop     Downloads          Music     Public   sample1   Templates
Documents   examples.desktop   Pictures  sample   sample2   Videos
```

Remove the file sample1

```
guru99@VirtualBox:~$ rm sample1
```

List directory , to check file has been deleted

```
guru99@VirtualBox:~$ ls
Desktop     Downloads          Music     Public   sample2    Videos
Documents   examples.desktop   Pictures  sample   Templates
guru99@VirtualBox:~$
```

Renaming Existing File

- For Renaming file use this command.
    - **mv filename newfilename.**

```
guru99@VirtualBox:~$ mv test test1
guru99@VirtualBox:~$ ls
Desktop    Downloads         Music     Public    test1
Documents  examples.desktop  Pictures  Templates  Videos
guru99@VirtualBox:~$
```

NOTE:

- By default, the password you entered for sudo is retained for 15 minutes per terminal. This eliminates the need of entering the password time and again.

- You only need root/sudo privileges, only if the command involves files or directories not owned by the user or group running the commands

Directory Manipulations

- Creating Directories

  Directories can be created on a Linux operating system using the following command-- **mkdir directoryname**

```
home@VirtualBox:~$ mkdir mydirectory
home@VirtualBox:~$ ls
Desktop    Downloads          Music        Pictures   Templates
Documents  examples.desktop   mydirectory  Public     Videos
home@VirtualBox:~$
```

- If you want to create a directory in a different location other than 'Home directory', you could use the following command – **mkdir /tmp/MUSIC**

Here MUSIC is a directory name which is created inside temp folder.

```
home@VirtualBox:~$ mkdir /tmp/MUSIC
home@VirtualBox:~$ ls /tmp
keyring-yCD2no  pulse-Ob9vyJcXyHZz  ssh-SSSsjczv1036      virtual-home.HaC7Mw
MUSIC           pulse-PKdhtXMmr18n  unity_support_test.1
home@VirtualBox:~$
```

- You can also create more than one directory at a time using the following command.

```
home@VirtualBox:~$ mkdir dir1 dir2 dir3
home@VirtualBox:~$ ls
Desktop   dir2  Documents   examples.desktop  Pictures   Templates
dir1      dir3  Downloads   Music             Public     Videos
home@VirtualBox:~$
```

Removing & Renaming Directories

- To remove a directory, use the command –

rmdir directoryname.

```
home@VirtualBox:~$ rmdir mydirectory
home@VirtualBox:~$ ls
Desktop   dir2   Documents   examples.desktop   Pictures   Templates
dir1      dir3   Downloads   Music              Public     Videos
home@VirtualBox:~$
```

Note:-

Ensure that there is no file / sub-directory under the directory that you want to delete. Delete the files/sub-directory first before deleting the parent directory.

```
home@VirtualBox:~$ rmdir Documents
rmdir: failed to remove `Documents': Directory not empty
home@VirtualBox:~$
```

Renaming Directory

The 'mv' (move) command (covered earlier) can also be used for renaming directories. Use the below-given format. **mv directoryname**

```
home@VirtualBox:~$ mv mydirectory newdirectory
home@VirtualBox:~$ ls
Desktop    Downloads          Music          Pictures   Templates
Documents  examples.desktop   newdirectory   Public     Videos
home@VirtualBox:~$
```

Other Important Command

- **Man Command:-**
  - Man stands for manual which is a reference book of a Linux operating system. It is similar to HELP file found in popular software.
  - To get help on any command that you do not understand, you can type:- **man**
  - The terminal would open the manual page for that command.
  - For an example, if we type ***man man*** and hit enter; terminal would give us information on man command.

```
guru99@VirtualBox:~$ man man
```

```
guru99@VirtualBox: ~
MAN(1)                          Manual pager utils                          MAN(1)

NAME
       man - an interface to the on-line reference manuals

SYNOPSIS
       man  [-C  file]  [-d]  [-D]  [--warnings[=warnings]]  [-R encoding] [-L
       locale] [-m system[,...]] [-M path] [-S list]  [-e  extension]  [-i|-I]
       [--regex|--wildcard]   [--names-only]  [-a]  [-u]  [--no-subpages]  [-P
       pager] [-r prompt] [-7] [-E encoding] [--no-hyphenation] [--no-justifi-
       cation]  [-p  string]  [-t]  [-T[device]]  [-H[browser]] [-X[dpi]] [-Z]
       [[section] page ...] ...
       man -k [apropos options] regexp ...
       man -K [-w|-W] [-S list] [-i|-I] [--regex] [section] term ...
       man -f [whatis options] page ...
       man -l [-C file] [-d] [-D] [--warnings[=warnings]]  [-R  encoding]  [-L
       locale]  [-P  pager]  [-r  prompt]  [-7] [-E encoding] [-p string] [-t]
       [-T[device]] [-H[browser]] [-X[dpi]] [-Z] file ...
       man -w|-W [-C file] [-d] [-D] page ...
       man -c [-C file] [-d] [-D] page ...
       man [-hV]

DESCRIPTION
 Manual page man(1) line 1 (press h for help or q to quit)
```

- **The History Command:-** History command shows all the commands that you have used in the past for the current terminal session. This can help you refer to the old commands you have entered and re-used them in your operations again.

```
guru99@VirtualBox:~$ history
    1  cat > sample
    2  cat sample
    3  cat sample ^a
    4  cat sample a
    5  cat sample | grep a
    6  cat sample | grep ^a
    7  useradd home
    8  useradd mycomputer
    9  sudo useradd mycomputer
   10  sudo adduser MyLinux
   11  sudo adduser mylinux
   12  vi scriptsample.sh
```

- **The clear command:-** This command clears all the clutter on the terminal and gives you a clean window to work on, just like when you launch the terminal.

```
  141  man
  142  3a
  143  man intro
  144  man ls
  145  man cat
  146  man man
  147  history
  148  146
  149  history 146
  150  history
  151  clear
  152  history
guru99@VirtualBox:~$ clear
```

The window gets cleared

```
guru99@VirtualBox:~$
```

- **chmod** file [flags] command :- This command change or set permission for the file defined by flags.
- **Permissiion :-** You can’t read ,write,edit or execute a file without permission.
-

```
Terminal
File  Edit  View  Terminal  Go  Help
rpowers@bionmr-01:~$ ls -l
total 412252
-rw-rw-r--  1 rpowers  bionmr     32844 Dec 22  2010 1oai.asa
-rw-rw-r--  1 rpowers  bionmr       512 Dec 22  2010 1oai.log
-rwxrw-r--  1 rpowers  bionmr     33220 Dec 22  2010 1oai.pdb
-rw-rw-r--  1 rpowers  bionmr      5332 Dec 22  2010 1oai.rsa
drwxrwxrwx  4 rpowers  bionmr      4096 Sep 15  2008 2D-Aromatic
-rw-rw-r--  1 rpowers  bionmr     27462 Dec 22  2010 2hba.asa
-rw-rw-r--  1 rpowers  bionmr       512 Dec 22  2010 2hba.log
-rwxrw-r--  1 rpowers  bionmr     27916 Dec 22  2010 2hba.pdb
-rw-rw-r--  1 rpowers  bionmr      4765 Dec 22  2010 2hba.rsa
drwxrwxrwx  2 rpowers  bionmr     20480 May 20 12:34 991A
```

Directory

Number of files in Directory

File Owner

Group Owner belongs to

Size of file in kilobytes

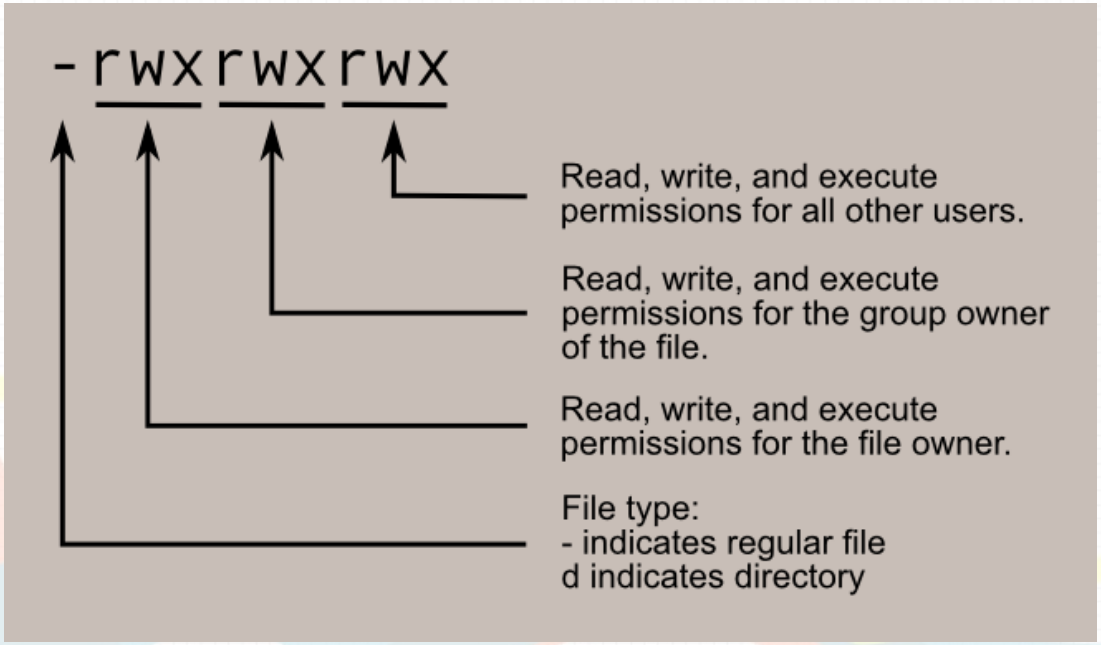
File Date or Time Stamp

Filename

- **Reading and understanding permissions:-**

```
Terminal
File  Edit  View  Terminal  Go  Help
rpowers@bionmr-01:~$ ls -l
total 412252
-rw-rw-r--  1 rpowers  bionmr     32844 Dec 22  2010 1oai.asa
-rw-rw-r--  1 rpowers  bionmr       512 Dec 22  2010 1oai.log
```

Permissions



Environment Variables in Linux

- Environment variables are dynamic values which affect the processes or programs on a computer. They exist in every operating system, but types may vary. Environment variables can be created, edited, saved, and deleted and give information about the system behaviour.

- Environment variables can change the way a software/programs behave.

- **PATH:-**This variable contains a colon (:)-separated list of directories in which your system looks for executable files.

```
guru99@VirtualBox:~$ echo $PATH
/usr/lib/lightdm/lightdm:/usr/local/sbin:/usr/local/bin:/usr
:/bin:/usr/games
```

- When you enter a command on terminal, the shell looks for the command in different directories mentioned in the $PATH variable. If the command is found, it executes. Otherwise, it returns with an error 'command not found'.
- **USER :-** User Name.
- **HOME :-** Default path to the user's home directory.
- **EDITOR:-** Path to the program which edits the content of files.
- **UID :-** User's unique ID.
- **TERM :-** Default terminal emulator.
- **SHELL :-** Shell being used by the user.

Accessing Variable values

- In order to determine value of a variable, use the command.

  echo $VARIABLE.
- **Echo $ VARIABLE:-** This command is used to display value of a variable
- Variables are- Case Sensitive. Make sure that you type the variable name in the right letter case otherwise you may not get the desired results.

```
home@VirtualBox:~$ echo $USER
home
home@VirtualBox:~$ echo $HOME
/home/home
home@VirtualBox:~$ echo $UID
1000
home@VirtualBox:~$ echo $TERM
xterm
home@VirtualBox:~$ echo $PATH
/usr/lib/lightdm/lightdm:/usr/local/sbin:/usr/local/bin
:/bin:/usr/games
home@VirtualBox:~$
```

- The '**env**' command displays **all the environment variables.**

```
guru99@VirtualBox:~$ env
SSH_AGENT_PID=8193
DBUS_STARTER_ADDRESS=unix:abstract=/tmp/dbus-3cJBjUA4bl
1018600004deb
GPG_AGENT_INFO=/tmp/keyring-lP9SZ1/gpg:0:1
TERM=xterm
SHELL=/bin/bash
XDG_SESSION_COOKIE=a5fb982bea8a8cb0299c2c9f00000007-134
WINDOWID=58720261
```

- You can create your own user defined variable, with syntax.

VARIABLE_NAME= variable_value

Note:-

- Again, bear in mind **that variables are case-sensitive** and usually they are created in upper case.

Define a new variable

```
ubuntu@ubuntu:~$ NEWVARIABLE=value1234
```

Check value of the newly created variable

```
ubuntu@ubuntu:~$ echo $NEWVARIABLE
value1234
```

TIP: Do not leave space between variable name and "=" sign you will get an error

```
ubuntu@ubuntu:~$ NEWVARIABLE = value1234
NEWVARIABLE: command not found
```

TIP: Do not forget the "$" sign when you check the value of variable

```
ubuntu@ubuntu:~$ echo NEWVARIABLE
NEWVARIABLE
```

Deleting Variables:

- The following syntax can be used to remove a Variable from the system:-- **unset variablename.**

Define a new variable

```
guru99@VirtualBox:~$ VARIABLE1=1234
```

Check value of the newly created variable

```
guru99@VirtualBox:~$ echo $VARIABLE1
1234
```

Use the 'unset' command

```
guru99@VirtualBox:~$ unset VARIABLE1
```

The variable value is removed

```
guru99@VirtualBox:~$ echo $VARIABLE1

guru99@VirtualBox:~$
```

This **would remove the Variable** and its value permanently.

Shell Scripts

- The shell script is a very useful tool in automating tasks in Linux OSes.
- It can also be used to combine utilities and create new commands.
- You can combine long and repetitive sequences of commands into one simple command.
- All scripts can be run without the need of compiling it, so the user will have a way of prototyping commands seamlessly.

Types of Shell

- The Bourne Shell
- The C Shell
- The Korn Shel
- The Bash Shell

The Bourne Shell

- The Bourne shell (sh), written by Steve Bourne at AT&T Bell Labs, is the original UNIX shell.

- It is the preferred shell for shell programming because of its compactness and speed.

- A Bourne shell drawback is that it lacks features for interactive use, such as the ability to recall previous commands (history).

- The Bourne shell is the Solaris OS default shell. It is the standard shell for Solaris system administration scripts.

  ✓ **For the Bourne shell the:**
  - Command full-path name is **/bin/sh** and **/sbin/sh**.
  - Non-root user default prompt is **$**.
  - Root user default prompt is **#**.

The C Shell(csh)

- Is a UNIX enhancement written by Bill Joy at the University of California at Berkeley.
- Incorporated features for interactive use, such as aliases and command history.
- Includes convenient programming features, such as built-in arithmetic and a C-like expression syntax.
- For the C shell the:
- Command full-path name is /bin/csh.
- Non-root user default prompt is hostname %.
- Root user default prompt is hostname #.

The Korn Shell(ksh)

- Was written by David Korn at AT&T Bell Labs
- Is a superset of the Bourne shell.
- Supports everything in the Bourne shell.
- Has interactive features comparable to those in the C shell.
- Includes convenient programming features like built-in arithmetic and C-like arrays, functions, and string-manipulation facilities.
- Is faster than the C shell.
- Runs scripts written for the Bourne shell.
- For the Korn shell the:
- Command full-path name is /bin/ksh.
- Non-root user default prompt is $.
- Root user default prompt is #.

The GNU Bourne-Again Shell(bash)

The bash (Bourne-Again Shell) is the default shell in most of the Linux distributions and OS X.

Is compatible to the Bourne shell.

Incorporates useful features from the Korn and C shells.

Has arrow keys that are automatically mapped for command recall and editing.

For the GNU Bourne-Again shell the:

- Command full-path name is /bin/bash.

Creating my first shell script

The bash (Bourne-Again Shell) is the default shell in most of the Linux distributions and OS X.

Is compatible to the Bourne shell.

Incorporates useful features from the Korn and C shells.

Has arrow keys that are automatically mapped for command recall and editing.

For the GNU Bourne-Again shell the:

- Command full-path name is /bin/bash.

First Shell Script

To Get the shell environment variable use this command:

echo $SHELL

Press Enter

```
snulab@HP-280-G3-MT:~$ echo $SHELL
/bin/bash
snulab@HP-280-G3-MT:~$
```

To display the information on terminal use this command:

echo ‘Hello Shell Scripting’

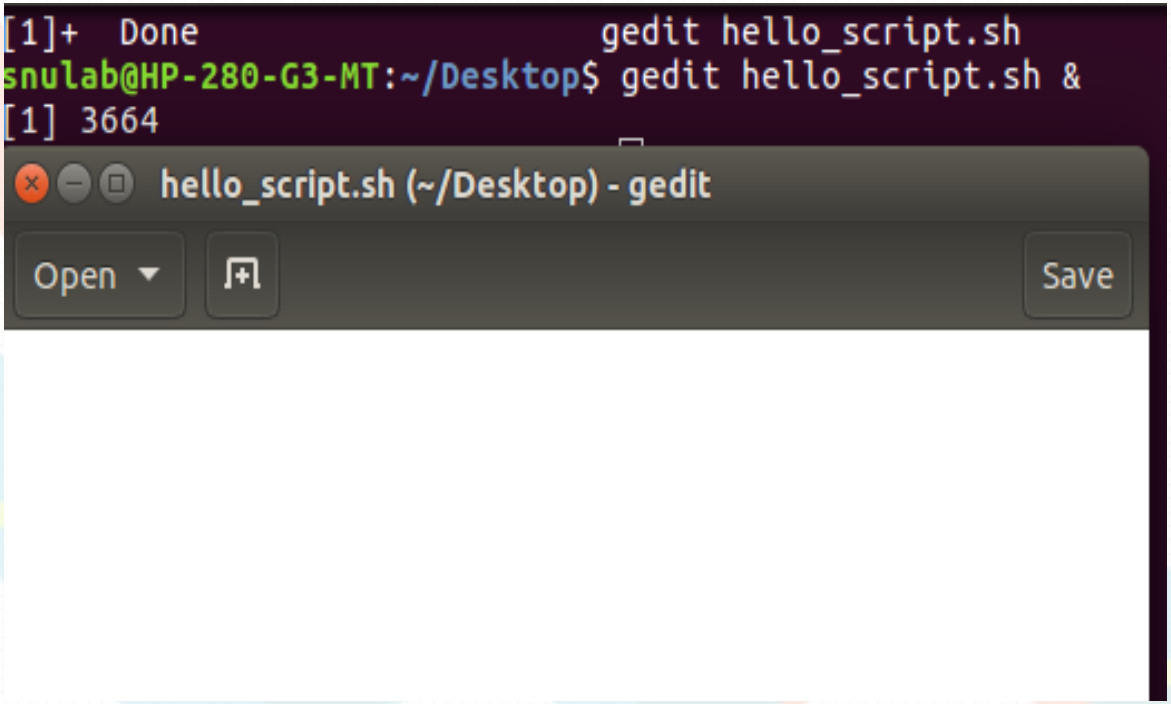
Press Enter

```
snulab@HP-280-G3-MT:~$ echo 'Hello Shell Scripting'
Hello Shell Scripting
snulab@HP-280-G3-MT:~$
```

Open your text editor by using this command:

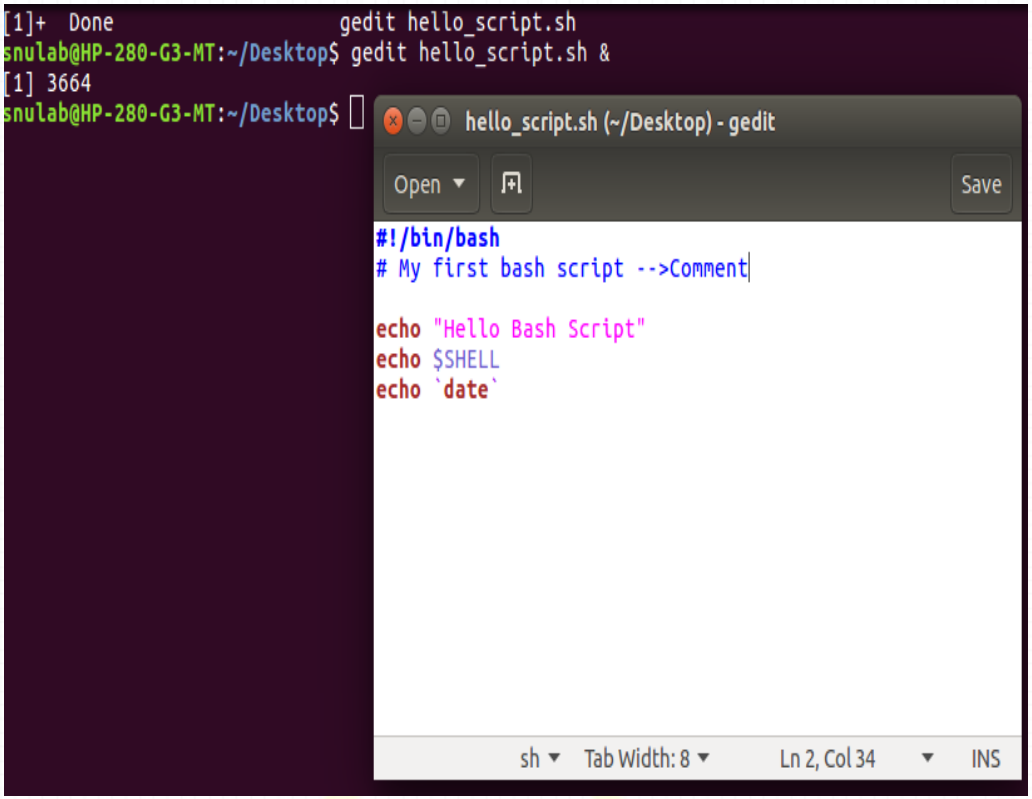
gedit hello_script.sh &

Press Enter



Write Script on text editor:

```
#!/bin/bash
# my First Bash Script
echo ”Hello Bash Script”
echo $SHELL
echo ‘date’
```



- A script may specify **#!/bin/bash** on the first line, **meaning** that the script should always be run with **bash**, rather than another shell. /**bin**/sh is an executable representing the system shell.

Change the permission of script file and make it executable by using this command:

- **chmod +x hello_script.sh**

```
snulab@HP-280-G3-MT:~/Desktop$ chmod +x hello_script.sh
```

When we make new script file then by default it has read and write permission. But if we want to execute them, then we should give execute permission by shown above.

To run the script use this command:

./hello_scipt.sh

```
snulab@HP-280-G3-MT:~/Desktop$ ./hello_script.sh
Hello Bash Script
/bin/bash
Mon Aug 6 12:43:00 IST 2018
snulab@HP-280-G3-MT:~/Desktop$
```